

Day 16

System Verilog: Assertions

Assertions for D – Flip flop:

```
module dff_with_assertions(
    input  logic clk, rst, d,
    output logic q
);
    always_ff @(posedge clk or posedge rst) begin
        if(rst) q <= 0;
        else    q <= d;
    end

    // Assertion 1: after reset, q must be 0
    property p_reset;
        @(posedge clk)
        $rose(rst) | => (q == 0);
    endproperty
    assert property(p_reset)
        $info("Success: q reset to 0 at time %t", $time);
    else $error("q not 0 after reset");

    // Assertion 2: without reset, q follows d after one cycle
    property p_dff;
        @(posedge clk)
        !rst | => (q == $past(d));
    endproperty
    assert property(p_dff)
        $info("Success: q followed d at time %t", $time);
    else $error("q did not follow d");

endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
    reg clk, d, rst;
    wire q;

    dff_with_assertions uut(.clk(clk), .d(d), .q(q), .rst(rst));
    initial begin
        $dumpfile("dff_sync_rst.vcd");
        $dumpvars(0, tb);
    end

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1; #10;
        d = 0; #10;
        d = 1; #10;
        rst = 0; #10;
        d = 1; #10;
    end
endmodule
```

```

d = 0; #10;
$finish;
end
endmodule

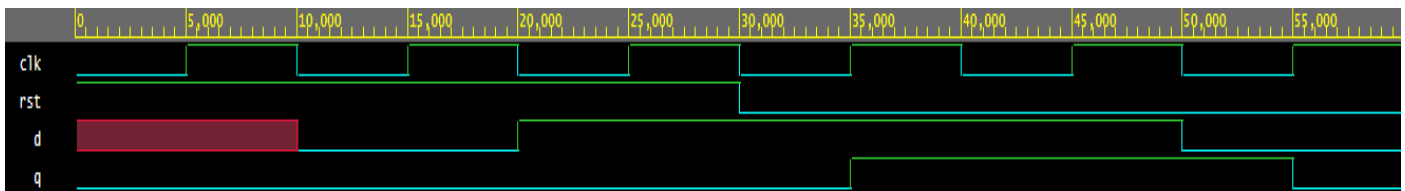
```

Output:

```

xmsim: *N,ASRTST (./design.sv,17): (time 15 NS) Assertion tb.uut.p_reset has passed
Success: q reset to 0 at time          15000
xmsim: *N,ASRTST (./design.sv,26): (time 45 NS) Assertion tb.uut.p_dff has passed
Success: q followed d at time          45000
xmsim: *N,ASRTST (./design.sv,26): (time 55 NS) Assertion tb.uut.p_dff has passed
Success: q followed d at time          55000
Simulation complete via $finish(1) at time 60 NS + 0

```



Deliberately causing an error

```

// Code your design here
module dff_with_assertions(
    input logic clk, rst, d,
    output logic q
);
    always_ff @(posedge clk) begin
        if(rst) q <= 1; // wrong — should be 0
        else q <= d;
    end

    // Assertion 1: after reset, q must be 0
    property p_reset;
        @(posedge clk)
        $rose(rst) | => (q == 0);
    endproperty
    assert property(p_reset)
        $info("Success: q reset to 0 at time %t", $time);
        else $error("Fail: q not 0 after reset");

    // Assertion 2: without reset, q follows d after one cycle
    property p_dff;
        @(posedge clk)
        !rst | => (q == $past(d));
    endproperty
    assert property(p_dff)
        $info("Success: q followed d at time %t", $time);
        else $error("Fail: q did not follow d");

endmodule

```

Output:

```

xmsim: *E,ASRTST (./design.sv,18): (time 15 NS) Assertion tb.uut.p_reset has failed
Fail: q not 0 after reset
xmsim: *N,ASRTST (./design.sv,26): (time 45 NS) Assertion tb.uut.p_dff has passed
Success: q followed d at time          45000
xmsim: *N,ASRTST (./design.sv,26): (time 55 NS) Assertion tb.uut.p_dff has passed
Success: q followed d at time          55000
Simulation complete via $finish(1) at time 60 NS + 0
./testbench.sv:23 $finish;

```

We can see that the assertion has failed here

Assertions for FIFO

```

module fifo_with_assertions #(
    parameter DEPTH = 8,
    parameter WIDTH = 8
)(
    input logic      clk, rst,
    input logic      wr_en, rd_en,
    input logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic      full, empty
);
    reg [WIDTH-1:0] mem [0:DEPTH-1];
    reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, count;
    // write
    always @(posedge clk) begin
        if(rst) wr_ptr <= 0;
        else if(wr_en && !full) begin
            mem[wr_ptr] <= din;
            wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
        end
    end
    //read
    always @(posedge clk) begin
        if(rst) rd_ptr <= 0;
        else if(rd_en && !empty) begin
            dout <= mem[rd_ptr];
            rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
        end
    end
    always @(posedge clk) begin
        if(rst) count <= 0;
        else begin
            case({wr_en & !full, rd_en & !empty})
                2'b10: count <= count+1;
                2'b01: count <= count-1;
                2'b11: count <= count;
                default: count <= count;
            endcase
        end
    end

    assign full = (count == DEPTH);
    assign empty = (count == 0);

```

// — Assertion 1: never write when full —————

```

property p_no_write_when_full;
    @(posedge clk) disable iff(rst)
    full |-> !wr_en;
endproperty
assert property(p_no_write_when_full)
else $error("Write attempted when FIFO full");

// — Assertion 2: never read when empty —————
property p_no_read_when_empty;
    @(posedge clk) disable iff(rst)
    empty |-> !rd_en;
endproperty
assert property(p_no_read_when_empty)
else $error("Read attempted when FIFO empty");

// — Assertion 3: full and empty never both true —————
property p_not_full_and_empty;
    @(posedge clk) disable iff(rst)
    !(full && empty);
endproperty
assert property(p_not_full_and_empty)
$info("Success: FIFO is not full and empty simulataneously");
else $error("Fail: FIFO full AND empty simultaneously ie impossible");

// — Assertion 4: after reset both flags correct —————
property p_reset_state;
    @(posedge clk)
    $rose(rst) |=> (empty == 1 && full == 0);
endproperty
assert property(p_reset_state)
$info("Success: Reset");
else $error("Fail: Wrong state after reset");

// — Assertion 5: full deasserts after a read —————
property p_full_clears_on_read;
    @(posedge clk) disable iff(rst)
    (full && rd_en) |=> !full;
endproperty
assert property(p_full_clears_on_read)
$info("Success: Full flag cleared the read");
else $error("Fail: Full flag did not clear after read");

// — Assertion 6: empty deasserts after a write —————
property p_empty_clears_on_write;
    @(posedge clk) disable iff(rst)
    (empty && wr_en) |=> !empty;
endproperty
assert property(p_empty_clears_on_write)
$info("Success: Empty flag cleared after write");
else $error("Fail: Empty flag did not clear after write");

```

endmodule

Testbench:

```
`timescale 1ns / 1ps
```

```

module tb;
    // 1. Parameters & Signals
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    reg clk, rst;
    reg wr_en, rd_en;
    reg [WIDTH-1:0] din;
    wire [WIDTH-1:0] dout;
    wire full, empty;
    wire [$clog2(DEPTH):0] count;

    // 2. Instantiate the FIFO
    fifo_with_assertions #(DEPTH, WIDTH) uut (
        .clk(clk), .rst(rst),
        .wr_en(wr_en), .rd_en(rd_en),
        .din(din), .dout(dout),
        .full(full), .empty(empty)
    );

    // 3. Clock Generation (10ns period)
    always #5 clk = ~clk;
    initial begin
        $dumpfile("fifo.vcd");
        $dumpvars(0, tb);
    end

    // 4. Test Stimulus
    initial begin
        // Initialize
        clk = 0; rst = 1; wr_en = 0; rd_en = 0; din = 0;
        #20 rst = 0; // Release reset

        // --- TEST 1: Fill the FIFO ---
        $display("--- Starting Fill Test ---");
        repeat (DEPTH) begin
            @(negedge clk);
            if (!full) begin
                wr_en = 1;
                din = $random % 256;
                $display("Writing: %h | Count expected to increase", din);
            end
        end

        @(negedge clk);
        wr_en = 0;
        #5;
        if (full) $display("FIFO is now FULL as expected.");

        // --- TEST 2: Overflow Attempt ---
        @(negedge clk);
        wr_en = 1; din = 8'hFF;
        #10;
        wr_en = 0;
        $display("Attempted write to full FIFO. Full flag is: %b", full);

        // --- TEST 3: Drain the FIFO ---
    end

```

```

$display("--- Starting Drain Test ---");
repeat (DEPTH) begin
    @(negedge clk);
    if (!empty) begin
        rd_en = 1;
    end
    @(posedge clk); #1; // Wait for data to clock out
    $display("Read: %h", dout);
end

@(negedge clk);
rd_en = 0;
#5;
if (empty) $display("FIFO is now EMPTY as expected.");

// --- TEST 4: Simultaneous Read and Write (Streaming) ---
$display("--- Starting Simultaneous Read/Write Test ---");

// First, put at least one item in so it's not empty
@(negedge clk);
wr_en = 1; rd_en = 0; din = 8'h11;

@(negedge clk);
// Now turn BOTH on
wr_en = 1;
rd_en = 1;
din = 8'h22; // Writing 22 while reading 11

repeat (3) begin
    @(negedge clk);
    din = din + 1; // Change data every cycle
    $display("Streaming - Count: %d | Full: %b | Empty: %b", count, full, empty);
end

@(negedge clk);
wr_en = 0;
rd_en = 0;

#50 $finish;
end
endmodule

```

Output:

```

xcelium> run
xmsim: *N,ASRTST (./design.sv,75): (time 15 NS) Assertion tb.uut.p_reset_state has passed
Success: Reset
--- Starting Fill Test ---
Writing: 24 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 25 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 81 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,93): (time 35 NS) Assertion tb.uut.p_empty_clears_on_write has passed
Success: Empty flag cleared after write
xmsim: *N,ASRTST (./design.sv,66): (time 35 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 09 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 45 NS) Assertion tb.uut.p_not_full_and_empty has passed

```

Success: FIFO is not full and empty simulataneously
Writing: 63 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 55 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 0d | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 65 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 8d | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 75 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 65 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 85 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Writing: 12 | Count expected to increase
xmsim: *N,ASRTST (./design.sv,66): (time 95 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
FIFO is now FULL as expected.
xmsim: *N,ASRTST (./design.sv,66): (time 105 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 115 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,50): (time 115 NS) Assertion tb.uut.p_no_write_when_full has failed
Write attempted when FIFO full
Attempted write to full FIFO. Full flag is: 1
--- Starting Drain Test ---
xmsim: *N,ASRTST (./design.sv,66): (time 125 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 24
xmsim: *N,ASRTST (./design.sv,84): (time 135 NS) Assertion tb.uut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,66): (time 135 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 81
xmsim: *N,ASRTST (./design.sv,66): (time 145 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 09
xmsim: *N,ASRTST (./design.sv,66): (time 155 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 63
xmsim: *N,ASRTST (./design.sv,66): (time 165 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 0d
xmsim: *N,ASRTST (./design.sv,66): (time 175 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 8d
xmsim: *N,ASRTST (./design.sv,66): (time 185 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 65
xmsim: *N,ASRTST (./design.sv,66): (time 195 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Read: 12
FIFO is now EMPTY as expected.
--- Starting Simultaneous Read/Write Test ---
xmsim: *N,ASRTST (./design.sv,66): (time 205 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 215 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,93): (time 225 NS) Assertion tb.uut.p_empty_clears_on_write has passed
Success: Empty flag cleared after write
xmsim: *N,ASRTST (./design.sv,66): (time 225 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Streaming - Count: z | Full: 0 | Empty: 0
xmsim: *N,ASRTST (./design.sv,66): (time 235 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Streaming - Count: z | Full: 0 | Empty: 0
xmsim: *N,ASRTST (./design.sv,66): (time 245 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Streaming - Count: z | Full: 0 | Empty: 0
xmsim: *N,ASRTST (./design.sv,66): (time 255 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 265 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 275 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 285 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 295 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,66): (time 305 NS) Assertion tb.uut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
Simulation complete via \$finish(1) at time 310 NS + 0
./testbench.sv:98 #50 \$finish;

All the cases have passed here